OUTAGE DISSECTIONS • DATABASE ARCHITECTURE • HIGH-SCALE OPERATIONS

Managing Production Databases: ProxySQL, Orchestrator & Anti-Flapping

Curated from Arpit Bhayani's Architecture Dissection Series • Core Focus: Database Topology Reliability

EXECUTIVE ARCHITECTURAL ABSTRACT

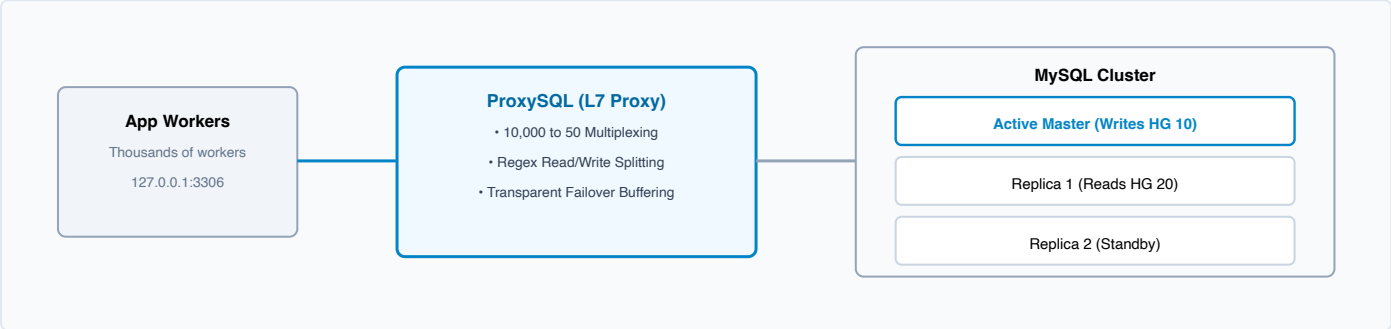
Operating hundreds of MySQL clusters at scale requires separating client traffic handling from database topology state. High-scale architectures (GitHub, Shopify) deploy **ProxySQL** as an L7 data plane proxy for connection multiplexing and query routing, paired with **GitHub Orchestrator** as an automated control plane for crash detection and replica promotion. However, during network flapping, automated failovers can churn out of control, requiring **Anti-Flapping Dampeners** to protect database integrity.

1. THE SCALING BOTTLENECK OF DIRECT DB CONNECTIONS

Direct application-to-database connections fail when traffic surges into tens of thousands of concurrent requests:

- **Thread Thrashing:** MySQL allocates memory and a thread per connection. At 5,000+ connections, thread contention causes kernel context-switching storms, dropping throughput by 80%.
- **Brittle Topology Coupling:** If applications connect directly to master hostnames, failing over requires DNS propagation or app server restarts, incurring minutes of downtime.
- **Read/Write Confusion:** Application code often routes reads to masters unnecessarily, starving write capacity.

2. TWO-TIER TOPOLOGY ARCHITECTURE



3. PROXYSQL MECHANICS: CONNECTION MULTIPLEXING & QUERY RULES

ProxySQL delivers low-latency SQL routing by operating as a high-performance C++ proxy:

Connection Multiplexing

ProxySQL decouples client frontend connections from backend server threads. 5,000 Puma workers share a tight pool of 50 persistent backend connections. Connections are returned to the pool immediately upon statement completion.

Dynamic Query Rules

Queries matching `^SELECT.*FOR UPDATE` or non-SELECT statements are mapped to Hostgroup 10 (Master). Standard `^SELECT` queries are load-balanced across Hostgroup 20 (Replicas) based on replication lag.

4. GITHUB ORCHESTRATOR: AUTOMATED CRASH DETECTION & RECOVERY

Orchestrator discovers and manages database topologies via continuous probing:

The Orchestrator Promotion Lifecycle

1. **Consensus Failure Probing:** Orchestrator nodes verify master death using cross-rack probes to eliminate false alarms caused by localized network drops.
2. **GTID-Based Candidate Selection:** Analyzes surviving replicas to select the node with the highest executed Global Transaction Identifier (GTID).
3. **Topology Restructuring:** Issues `STOP SLAVE`, sets `read_only = 0` on the promoted candidate, and repoints sibling replicas using `CHANGE MASTER TO MASTER_AUTO_POSITION = 1`.
4. **ProxySQL Re-targeting:** Executes an automated REST hook to update ProxySQL's `mysql_servers` table, shifting the write hostgroup to the new master without client connection drops.

5. THE DANGER OF TOPOLOGY FLAPPING

When a server experiences flapping hardware or intermittent packet loss, naive automation triggers disaster:

```
T=0s:  Master A drops heartbeats → Orchestrator promotes Replica B.
T=15s: Master A re-awakens → receives residual writes (split-brain).
T=30s: Replica B network blips → Orchestrator promotes Replica C.
T=60s: Three distinct nodes have acted as master; binlogs are irreconcilably diverged!
```

6. ANTI-FLAPPING DAMPENER ENGINEERING

Orchestrator prevents churn through three strict anti-flapping guardrails:

Guardrail 1: RecoveryGracePeriod (Downtime Cooldown)

Once Orchestrator executes a failover on a given cluster, automated failover is locked out for `RecoveryGracePeriod` (typically 10 minutes). If another failure occurs within this window, automation refuses to act and alerts on-call engineers.

Guardrail 2: Enforced Fencing & Ex-Master Demotion

A deposed master is immediately fenced via network iptables or AWS security group revocation. If it boots up again, its startup scripts force `read_only = 1`; it is never allowed to reclaim leadership automatically.

7. ARCHITECTURE COMPARISON: MANAGED DB VS. BARE CLUSTER

Architecture Model	Failover Latency	App Transparency	Flapping Risk
DNS-Based Failover	30–120 seconds (TTL caching)	Poor (Throws connection errors)	High (Split DNS propagation)
ProxySQL + Orchestrator	**Sub-second (200–500ms)**	**Seamless (In-flight pause)**	**Zero (Grace periods & GTID)**
Cloud-Native (Aurora / Spanner)	10–30s / Zero (Paxos quorums)	High (Handled at storage layer)	Zero (Distributed consensus)

8. PRODUCTION ENGINEERING TAKEAWAYS

1. **Separate Data Plane from Control Plane:** ProxySQL handles the high-throughput query data path; Orchestrator handles topology control. Decoupling them allows database failovers without restarting applications.
2. **Automate with Brakes:** Automated self-healing must include rate limiters. Without an anti-flapping grace period, an automated tool will enthusiastically destroy your cluster during network instability.
3. **Rely on GTID for Zero-Downtime Repointing:** Never use binlog file and offset coordinates for failovers at scale. GTID enables deterministic, error-free replica adoption across any topology hierarchy.